Large and Fast: Exploiting Memory Hierarchy: Part I

CSE 4305: Computer Organization and Architecture

Mohammad Ishrak Abedin
Lecturer, Department of CSE

09 February 2026

Islamic University of Technology

Table of Contents

Introduction
Memory Technologies
The Basics of Cache
Measuring and Improving Cache Performance
Dependable Memory Hierarchy

”

Ideally one would desire an indefinitely large memory capacity such that any particular . . . word would be immediately available. . . . We are . . . forced to recognize the possibility of constructing a hierarchy of memories, each of which has greater capacity than the preceding but which is less quickly accessible.
Ideally one would desire an indefinitely large memory capacity such that any particular . . . word would be immediately available. . . . We are . . . forced to recognize the possibility of constructing a hierarchy of memories, each of which has greater capacity than the preceding but which is less quickly accessible.

Preliminary Discussion of the Logical Design of an Electronic Computing Instrument, 1946

A. W. Burks, H. H. Goldstine, and J. von Neumann

Introduction

Principle of Locality

- Programs access a small portion of their address space at any time

Principle of Locality

- Programs access a small portion of their address space at any time
- Temporal locality
  - Items accessed recently are likely to be accessed again soon
  - e.g., instructions in a loop, induction variables

Principle of Locality

- Programs access a small portion of their address space at any time
- Temporal locality
  - Items accessed recently are likely to be accessed again soon
  - e.g., instructions in a loop, induction variables
- Spatial locality
  - Items near those accessed recently are likely to be accessed soon
  - e.g., sequential instruction access, array data

Taking Advantage of Locality

- Hierarchical memory design

Taking Advantage of Locality

- Hierarchical memory design
- Store everything on disk

Taking Advantage of Locality

- Hierarchical memory design
- Store everything on disk
- Copy recently accessed (and nearby) items from disk to smaller DRAM memory (main memory)

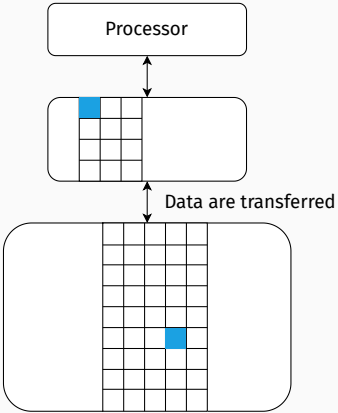
Taking Advantage of Locality

- Hierarchical memory design
- Store everything on disk
- Copy recently accessed (and nearby) items from disk to smaller DRAM memory (main memory)
- Copy more recently accessed (and nearby) items from DRAM to smaller SRAM memory (cache memory attached to the CPU)

Basic Structure of a Memory Hierarchy

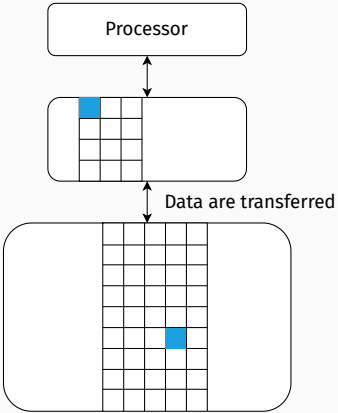
Speed		Size	Cost($/bit)	Current Technology
	Processor			
Fastest	Memory	Smallest	Highest	SRAM
↑	Memory	↓	↑	DRAM
Slowest	Memory	Biggest	Lowest	Magnetic disk or Flash memory

Memory Hierarchy Levels and Terminologies



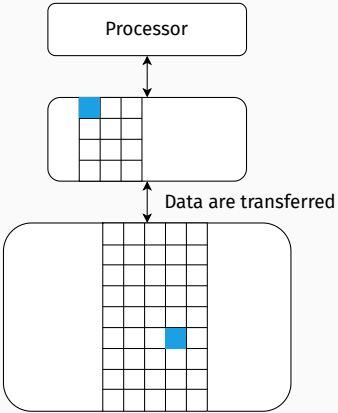
- Block (line): unit of copying, may be multiple words

Memory Hierarchy Levels and Terminologies



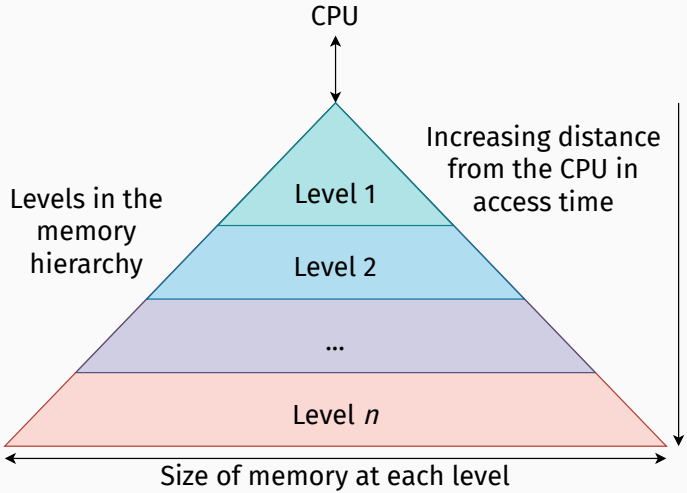
- Block (line): unit of copying, may be multiple words
- If accessed data is present in upper-level —
  - Hit: access satisfied by upper level
  - Hit ratio: hits/accesses

Memory Hierarchy Levels and Terminologies



- Block (line): unit of copying, may be multiple words
- If accessed data is present in upper-level —
  - Hit: access satisfied by upper level
  - Hit ratio: hits/accesses
- If accessed data is absent —
  - Miss: block copied from lower level
    - Time taken: miss penalty
    - Miss ratio: $\frac{misses}{accesses} = 1 - hit\ ratio$
  - Then accessed data supplied to the upper-level

Memory Hierarchy Levels Structure



Memory Technologies

Memory Technologies

- Static RAM (SRAM)
  - Access time: 0.5ns – 2.5ns, Cost: $500 – $1000 per GB

Memory Technologies

- Static RAM (SRAM)
  - Access time: 0.5ns – 2.5ns, Cost: $500 – $1000 per GB
- Dynamic RAM (DRAM)
  - Access time: 50ns – 70ns, Cost: $3 – $6 per GB

Memory Technologies

- Static RAM (SRAM)
  - Access time: 0.5ns – 2.5ns, Cost: $500 – $1000 per GB
- Dynamic RAM (DRAM)
  - Access time: 50ns – 70ns, Cost: $3 – $6 per GB
- Flash
  - Access time: 5,000ns – 50,000ns, Cost: $0.06 – $0.12 per GB

Memory Technologies

- Static RAM (SRAM)
  - Access time: 0.5ns – 2.5ns, Cost: $500 – $1000 per GB
- Dynamic RAM (DRAM)
  - Access time: 50ns – 70ns, Cost: $3 – $6 per GB
- Flash
  - Access time: 5,000ns – 50,000ns, Cost: $0.06 – $0.12 per GB
- Magnetic Disk
  - Access time: 5,000,000ns – 20,000,000ns, Cost: $0.01 – $0.02 per GB

Memory Technologies

- Static RAM (SRAM)
  - Access time: 0.5ns – 2.5ns, Cost: $500 – $1000 per GB
- Dynamic RAM (DRAM)
  - Access time: 50ns – 70ns, Cost: $3 – $6 per GB
- Flash
  - Access time: 5,000ns – 50,000ns, Cost: $0.06 – $0.12 per GB
- Magnetic Disk
  - Access time: 5,000,000ns – 20,000,000ns, Cost: $0.01 – $0.02 per GB

Ideal Memory

- Access time of SRAM
- Capacity and cost/GB of disk

SRAM Technology

SRAM is an integrated memory array, typically with a single read/write access port

SRAM Technology

SRAM is an integrated memory array, typically with a single read/write access port

- Offers fixed access time to any location, read and write times may differ

SRAM Technology

SRAM is an integrated memory array, typically with a single read/write access port

- Offers fixed access time to any location, read and write times may differ
- No refresh is required, access time is close to the cycle time

SRAM Technology

SRAM is an integrated memory array, typically with a single read/write access port

- Offers fixed access time to any location, read and write times may differ
- No refresh is required, access time is close to the cycle time
- Each bit uses 6–8 transistors, ensuring stable reads

SRAM Technology

SRAM is an integrated memory array, typically with a single read/write access port

- Offers fixed access time to any location, read and write times may differ
- No refresh is required, access time is close to the cycle time
- Each bit uses 6–8 transistors, ensuring stable reads
- Consumes very little power in standby

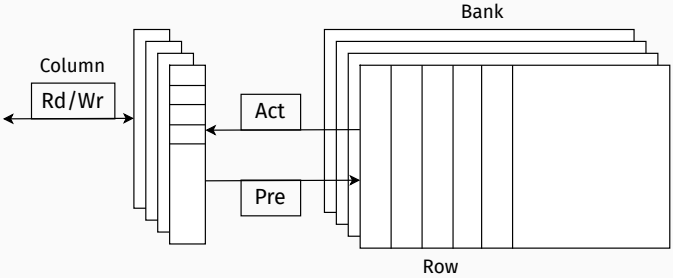
SRAM Technology

SRAM is an integrated memory array, typically with a single read/write access port

- Offers fixed access time to any location, read and write times may differ
- No refresh is required, access time is close to the cycle time
- Each bit uses 6–8 transistors, ensuring stable reads
- Consumes very little power in standby
- Once used as separate cache chips, SRAM caches are now integrated on the processor due to Moore's Law

DRAM Technology

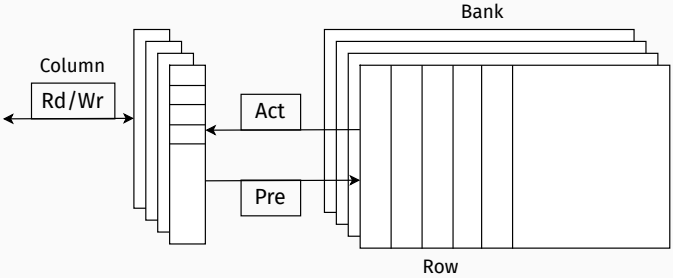
Data stored as a charge in a capacitor



DRAM Technology

Data stored as a charge in a capacitor

- Single transistor used to access the charge

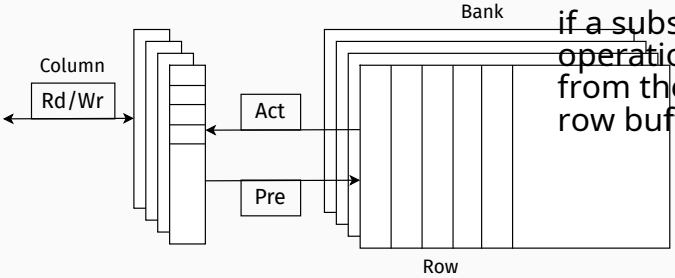


DRAM Technology

Data stored as a charge in a capacitor

- Single transistor used to access the charge
- Must periodically be refreshed
  - Read contents and write back
  - Performed on a DRAM row

* every dram bank contains one row buffer
* when cpu asks for data in a specific cell, full row gets activated
* full row is copied into the row buffer
* then the data in the required cell is returned



if a subsequent operation wants data from the same row -> row buffer hit

Advanced DRAM Organization I

- Bits in a DRAM are organized as a rectangular array
  - DRAM accesses an entire row
  - Burst mode: supply successive words from a row with reduced latency

Advanced DRAM Organization I

- Bits in a DRAM are organized as a rectangular array
  - DRAM accesses an entire row
  - Burst mode: supply successive words from a row with reduced latency
- Synchronous DRAM (SDRAM)
  - DRAMS with added clocks
  - Eliminates the time for the memory and processor to synchronize

Advanced DRAM Organization I

- Bits in a DRAM are organized as a rectangular array
  - DRAM accesses an entire row
  - Burst mode: supply successive words from a row with reduced latency
- Synchronous DRAM (SDRAM)
  - DRAMS with added clocks
  - Eliminates the time for the memory and processor to synchronize
- Double Data Rate (DDR) SDRAM
  - Transfer on both rising and falling clock edges — twice as much bandwidth
  - A DDR4-3200 DRAM can do 3,200 million transfers per second, which means it has a 1600-MHz clock

Advanced DRAM Organization II

- Address Interleaving
  - Internally organized to read or write from multiple banks, each having its own row buffer
  - With four banks, there is just one access time and then accesses rotate between the four banks to supply four times the bandwidth

Sequential memory blocks go across different banks so that multiple banks can operate in parallel since each bank has only one row buffer

Advanced DRAM Organization II

- Address Interleaving
  - Internally organized to read or write from multiple banks, each having its own row buffer
  - With four banks, there is just one access time and then accesses rotate between the four banks to supply four times the bandwidth
- Dual Inline Memory Module (DIMM)
  - Memory for servers, commonly sold on small boards
  - Typically contain 4–16 DRAMs, organized to be 8 bytes wide for server systems

DRAM Performance Factors

- Row Buffer
  - Allows several words to be read and refreshed in parallel

DRAM Performance Factors

- Row Buffer
  - Allows several words to be read and refreshed in parallel
- Synchronous DRAM
  - Allows for consecutive accesses in bursts without needing to send each address
  - Improves bandwidth

DRAM Performance Factors

- Row Buffer
  - Allows several words to be read and refreshed in parallel
- Synchronous DRAM
  - Allows for consecutive accesses in bursts without needing to send each address
  - Improves bandwidth
- DRAM Banking
  - Allows simultaneous access to multiple DRAMs
  - Improves bandwidth

DRAM Generations

Year Introduced	Chip Size	$ per GiB	Total Access Time to a New Row/Column	Average Column Access Time to Existing Row
1980	64 Kibibit	$6,480,000	250 ns	150 ns
1983	256 Kibibit	$1,980,000	185 ns	100 ns
1985	1 Mebibit	$720,000	135 ns	40 ns
1989	4 Mebibit	$128,000	110 ns	40 ns
1992	16 Mebibit	$30,000	90 ns	30 ns
1996	64 Mebibit	$9,000	60 ns	12 ns
1998	128 Mebibit	$900	60 ns	10 ns
2000	256 Mebibit	$840	55 ns	7 ns
2004	512 Mebibit	$150	50 ns	5 ns
2007	1 Gibibit	$40	45 ns	1.25 ns
2010	2 Gibibit	$13	40 ns	1 ns
2012	4 Gibibit	$5	35 ns	0.8 ns
2015	8 Gibibit	$7	30 ns	0.6 ns
2018	16 Gibibit	$6	25 ns	0.4 ns

Flash Storage

- Non-volatile semiconductor storage
  - A type of electrically erasable programmable read-only memory (EEPROM)

Flash Storage

- Non-volatile semiconductor storage
  - A type of electrically erasable programmable read-only memory (EEPROM)
  - 100×–1000× faster than disks

Flash Storage

- Non-volatile semiconductor storage
  - A type of electrically erasable programmable read-only memory (EEPROM)
  - 100×–1000× faster than disks
  - Smaller, lower power, more rugged

Flash Storage

- Non-volatile semiconductor storage
  - A type of electrically erasable programmable read-only memory (EEPROM)
  - 100×–1000× faster than disks
  - Smaller, lower power, more rugged
  - But more $/GB (between disk and DRAM)

Flash Types

- NOR Flash: Each memory cell connected directly to bit lines (like NOR gates)
  - Very fast read access
  - Random byte-level access
  - Slower erase and write
  - Larger cell size → lower density
  - More expensive per bit
  - Commonly used in BIOS/UEFI/microcontroller firmware, boot ROMs, etc.

Flash Types

- NOR Flash: Each memory cell connected directly to bit lines (like NOR gates)
  - Very fast read access
  - Random byte-level access
  - Slower erase and write
  - Larger cell size → lower density
  - More expensive per bit
  - Commonly used in BIOS/UEFI/microcontroller firmware, boot ROMs, etc.
- NAND Flash: Cells connected in series (like NAND gates)
  - Data is accessed in pages and erased in blocks, no true random byte access
  - Very high storage density
  - Much cheaper per bit
  - Fast sequential read/write but slower random access
  - Requires error correction (ECC)
  - Commonly used in SSDs, USB flash drives, SD cards, eMMC/UFS storage, etc.

Wear Out of Flash Memory Bits

- Writes can wear out flash memory bits

Wear Out of Flash Memory Bits

- Writes can wear out flash memory bits
- Wear leveling: Include a controller to spread the writes by remapping blocks to less trodden blocks

Wear Out of Flash Memory Bits

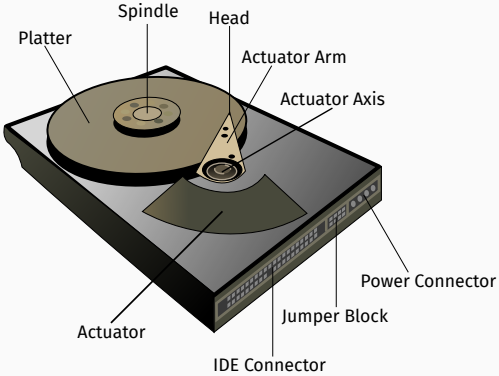
- Writes can wear out flash memory bits
- Wear leveling: Include a controller to spread the writes by remapping blocks to less trodden blocks
- Wear leveling lowers potential performance but required for longevity (unless higher-level software monitors block wear)

Wear Out of Flash Memory Bits

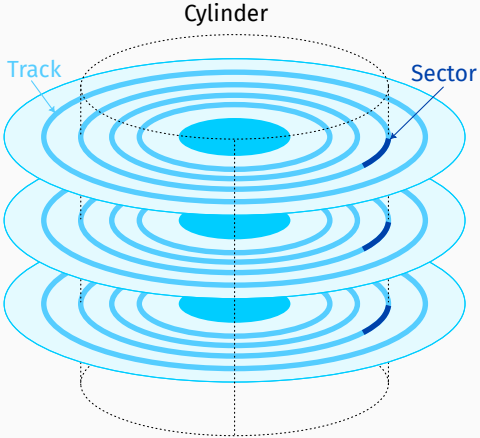
- Writes can wear out flash memory bits
- Wear leveling: Include a controller to spread the writes by remapping blocks to less trodden blocks
- Wear leveling lowers potential performance but required for longevity (unless higher-level software monitors block wear)
- Such controllers can also improve yield by mapping out incorrectly manufactured memory cells

Disk Storage

Non-volatile, rotating magnetic storage



Source: Wikipedia (by I, Surachit, CC BY-SA 3.0)



Physical Structure and Data Organization

- Physical Structure
  - Consists of collection of platters rotating on a spindle
  - Rotation speeds typically 5,400 to 15,000 RPM
  - Platters covered with magnetic recording material
  - Movable arm positions read-write head over surface
  - Drive permanently sealed to allow close proximity to surface

Physical Structure and Data Organization

- Physical Structure
  - Consists of collection of platters rotating on a spindle
  - Rotation speeds typically 5,400 to 15,000 RPM
  - Platters covered with magnetic recording material
  - Movable arm positions read-write head over surface
  - Drive permanently sealed to allow close proximity to surface
- Data Organization
  - Surfaces divided into concentric circles called tracks
  - Tracks divided into sectors containing information
  - Sectors typically 512 to 4096 bytes in size
  - Sector structure includes

number	gap	data with ECC	gap	...

Physical Structure and Data Organization

- Physical Structure
  - Consists of collection of platters rotating on a spindle
  - Rotation speeds typically 5,400 to 15,000 RPM
  - Platters covered with magnetic recording material
  - Movable arm positions read-write head over surface
  - Drive permanently sealed to allow close proximity to surface
- Data Organization
  - Surfaces divided into concentric circles called tracks
  - Tracks divided into sectors containing information
  - Sectors typically 512 to 4096 bytes in size
  - Sector structure includes

number	gap	data with ECC	gap	...

- Head Movement and Cylinders
  - Heads for each surface connected to move in conjunction
  - Cylinder refers to all tracks under heads at a given point

Access Process

OS directs disk through three-stage access process —

Access Process

OS directs disk through three-stage access process —

- Stage 1: Seek Time
  - Positioning the head over the proper track is called a seek
  - Average seek time is sum of all possible seeks divided by count
  - Advertised times range from 3 ms to 13 ms
  - Actual average often lower due to locality of references

Access Process

OS directs disk through three-stage access process —

- Stage 1: Seek Time
  - Positioning the head over the proper track is called a seek
  - Average seek time is sum of all possible seeks divided by count
  - Advertised times range from 3 ms to 13 ms
  - Actual average often lower due to locality of references
- Stage 2: Rotational Latency
  - Wait time for desired sector to rotate under head
  - Termed rotational latency or rotational delay
  - Average latency is time for half a rotation (Approximately 5.6 ms at 5400 RPM)

Access Process

OS directs disk through three-stage access process —

- Stage 1: Seek Time
  - Positioning the head over the proper track is called a seek
  - Average seek time is sum of all possible seeks divided by count
  - Advertised times range from 3 ms to 13 ms
  - Actual average often lower due to locality of references
- Stage 2: Rotational Latency
  - Wait time for desired sector to rotate under head
  - Termed rotational latency or rotational delay
  - Average latency is time for half a rotation (Approximately 5.6 ms at 5400 RPM)
- Stage 3: Transfer Time
  - Time to transfer a block of bits (150 to 250 MB/sec in 2020)
  - Function of sector size, rotation speed, and recording density
  - Built-in controller cache allows higher transfer rates

Disk Access Example

- Calculate the average read time for a disk access given the following performance parameters:
  - Data size: 512 bytes
  - Rotation speed: 15,000 RPM
  - Transfer rate: 100 MB/s
  - Average seek time: 4 ms
  - Controller overhead: 0.2 ms

Disk Access Example

- Calculate the average read time for a disk access given the following performance parameters:
  - Data size: 512 bytes
  - Rotation speed: 15,000 RPM
  - Transfer rate: 100 MB/s
  - Average seek time: 4 ms
  - Controller overhead: 0.2 ms
- Solution:

Disk Access Example

- Calculate the average read time for a disk access given the following performance parameters:
  - Data size: 512 bytes
  - Rotation speed: 15,000 RPM
  - Transfer rate: 100 MB/s
  - Average seek time: 4 ms
  - Controller overhead: 0.2 ms
- Solution:
  - Rotational latency, $T_{rotation} = 0.5 \times \frac{1}{15{,}000/60} ms = 0.5 \times \frac{1}{250} ms = 2ms$

Disk Access Example

- Calculate the average read time for a disk access given the following performance parameters:
  - Data size: 512 bytes
  - Rotation speed: 15,000 RPM
  - Transfer rate: 100 MB/s
  - Average seek time: 4 ms
  - Controller overhead: 0.2 ms
- Solution:
  - Rotational latency, $T_{rotation} = 0.5 \times \frac{1}{15{,}000/60} ms = 0.5 \times \frac{1}{250} ms = 2ms$
  - Transfer time, $T_{transfer} = \frac{512}{100 \times 10^6} s = 5.12 \times 10^{-6} s = 0.005ms$

Disk Access Example

- Calculate the average read time for a disk access given the following performance parameters:
  - Data size: 512 bytes
  - Rotation speed: 15,000 RPM
  - Transfer rate: 100 MB/s
  - Average seek time: 4 ms
  - Controller overhead: 0.2 ms
- Solution:
  - Rotational latency, $T_{rotation} = 0.5 \times \frac{1}{15{,}000/60} ms = 0.5 \times \frac{1}{250} ms = 2ms$
  - Transfer time, $T_{transfer} = \frac{512}{100 \times 10^6} s = 5.12 \times 10^{-6} s = 0.005ms$
  - Total average read time,
    $T_{access} = T_{seek} + T_{rotation} + T_{transfer} + T_{controller} = (4 + 2 + 0.005 + 0.2)ms = 6.205ms$

Disk Access Example

- Calculate the average read time for a disk access given the following performance parameters:
  - Data size: 512 bytes
  - Rotation speed: 15,000 RPM
  - Transfer rate: 100 MB/s
  - Average seek time: 4 ms
  - Controller overhead: 0.2 ms
- Solution:
  - Rotational latency, $T_{rotation} = 0.5 \times \frac{1}{15{,}000/60} ms = 0.5 \times \frac{1}{250} ms = 2ms$
  - Transfer time, $T_{transfer} = \frac{512}{100 \times 10^6} s = 5.12 \times 10^{-6} s = 0.005ms$
  - Total average read time,
    $T_{access} = T_{seek} + T_{rotation} + T_{transfer} + T_{controller} = (4 + 2 + 0.005 + 0.2)ms = 6.205ms$
- If actual average seek time is $1ms$, average read time would be $3.205ms$

Modern Addressing and Trade-offs

- Interfaces now organize disks sequentially like tapes

Modern Addressing and Trade-offs

- Interfaces now organize disks sequentially like tapes
- Logical blocks ordered in serpentine fashion

Modern Addressing and Trade-offs

- Interfaces now organize disks sequentially like tapes
- Logical blocks ordered in serpentine fashion
- Slower access than flash ($1000\times$) or DRAM ($100,000\times$)

Modern Addressing and Trade-offs

- Interfaces now organize disks sequentially like tapes
- Logical blocks ordered in serpentine fashion
- Slower access than flash ($1000\times$) or DRAM ($100,000\times$)
- Cheaper per bit with very high storage capacity

Modern Addressing and Trade-offs

- Interfaces now organize disks sequentially like tapes
- Logical blocks ordered in serpentine fashion
- Slower access than flash ($1000\times$) or DRAM ($100,000\times$)
- Cheaper per bit with very high storage capacity
- Non-volatile like flash but no write wear-out problem

The Basics of Cache

Cache Memory

- Cache memory: The level of the memory hierarchy between the processor and main memory (also used to refer to any storage managed to take advantage of locality of access)

Cache Memory

- Cache memory: The level of the memory hierarchy between the processor and main memory (also used to refer to any storage managed to take advantage of locality of access)
- Example: Considering a simple cache, where a single cache block (line) consists of a single word, given access of X_n after $X_1, X_2, \ldots, X_{n-1}$ —

Cache Memory

- Cache memory: The level of the memory hierarchy between the processor and main memory (also used to refer to any storage managed to take advantage of locality of access)
- Example: Considering a simple cache, where a single cache block (line) consists of a single word, given access of X_n after $X_1, X_2, \ldots, X_{n-1}$ —

X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_3

Before reference to X_n

Cache Memory

- Cache memory: The level of the memory hierarchy between the processor and main memory (also used to refer to any storage managed to take advantage of locality of access)
- Example: Considering a simple cache, where a single cache block (line) consists of a single word, given access of X_n after $X_1, X_2, \ldots, X_{n-1}$ —

X_4	X_4
X_1	X_1
X_{n-2}	X_{n-2}
X_{n-1}	X_{n-1}
X_2	X_2
	X_n
X_3	X_3
Before reference to X_n	After reference to X_n

Cache Memory

- Cache memory: The level of the memory hierarchy between the processor and main memory (also used to refer to any storage managed to take advantage of locality of access)
- Example: Considering a simple cache, where a single cache block (line) consists of a single word, given access of X_n after $X_1, X_2, \ldots, X_{n-1}$ —

X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_3

Before reference to X_n

X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_n
X_3

After reference to X_n

- Two questions:

Cache Memory

- Cache memory: The level of the memory hierarchy between the processor and main memory (also used to refer to any storage managed to take advantage of locality of access)
- Example: Considering a simple cache, where a single cache block (line) consists of a single word, given access of X_n after $X_1, X_2, \ldots, X_{n-1}$ —

X_4	X_4
X_1	X_1
X_{n-2}	X_{n-2}
X_{n-1}	X_{n-1}
X_2	X_2
	X_n
X_3	X_3
Before reference to X_n	After reference to X_n

- Two questions:
  1. How do we know if a data is present?

Cache Memory

- Cache memory: The level of the memory hierarchy between the processor and main memory (also used to refer to any storage managed to take advantage of locality of access)
- Example: Considering a simple cache, where a single cache block (line) consists of a single word, given access of X_n after $X_1, X_2, \ldots, X_{n-1}$ —

X_4	X_4
X_1	X_1
X_{n-2}	X_{n-2}
X_{n-1}	X_{n-1}
X_2	X_2
	X_n
X_3	X_3
Before reference to X_n	After reference to X_n

- Two questions:
  1. How do we know if a data is present?
  2. How do we find it if it is present?

Direct Mapped Cache

- Location determined by the memory address

Direct Mapped Cache

- Location determined by the memory address
- Direct mapped: only one choice

$$\textit{Block Index} = (\textit{Block Address}) \textit{ modulo } (\textit{Number of blocks in the cache})$$

Direct Mapped Cache

```
N % 2^k = k lowest bits
of N
10110 % 2^3 = 110
```

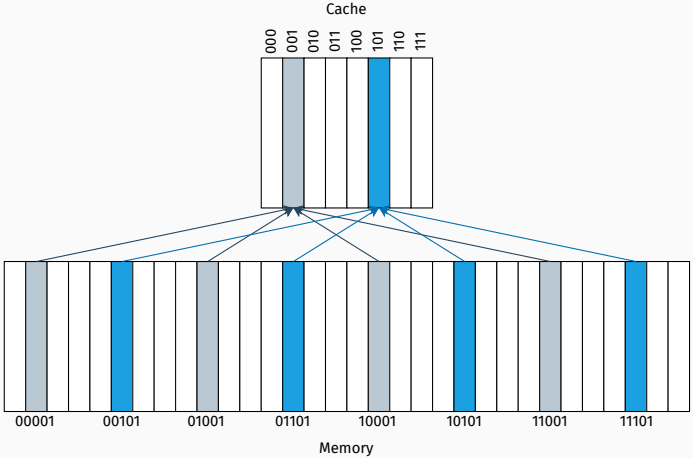
- Location determined by the memory address
- Direct mapped: only one choice

  Block Index = (Block Address) modulo (Number of blocks in the cache)

- If the number of blocks is a power of two, we can simply use lower-order address bits

Direct mapping determines which cache line a retrieved memory block is placed into using a modulo ops. if the number of blocks in the cache is the k-th power of 2 then the result of x 2^k is simply the lowest k bits so it doesnt strain the hardware

Direct Mapped Cache Example



A direct-mapped cache with eight entries showing memory words between 0 and 31 that map to the same blocks

Tags and Valid Bits

- How do we know which particular block is stored in a cache location?

Tags and Valid Bits

```
1101 |010 | 1000
Physical Adress = tag + idx
1101010
Cache Line also has a valid bit which
tells that the data is not garbage
```

- How do we know which particular block is stored in a cache location?
  - Store block address as well as data
  - Only the higher-order bits are necessary (non-redundant)
  - Called the tag

The CPU generates a memory address which the cache interprets as | Tag | Index | Offset |.
The index selects the cache line containing a block of words, and the offset selects the specific word inside that block. The tag contains the upper bits of the memory address that identify the memory block. The cache data is valid if the tag from the CPU address matches the stored tag in the cache line and the valid bit is set.

Tags and Valid Bits

- How do we know which particular block is stored in a cache location?
  - Store block address as well as data
  - Only the higher-order bits are necessary (non-redundant)
  - Called the tag
- What if there is no valid data in a location?

Tags and Valid Bits

- How do we know which particular block is stored in a cache location?
  - Store block address as well as data
  - Only the higher-order bits are necessary (non-redundant)
  - Called the tag
- What if there is no valid data in a location?
  - Use valid bit — 1 means present (valid), 0 means absent (invalid)
  - Initially set to 0

Cache Example

Cache

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Look-ups

Dec Word Addr	Binary Addr	Hit/Miss	Cache Block

Cache Example

Cache

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10	Mem(10110)
111	N		

Look-ups

Dec Word Addr	Binary Addr	Hit/Miss	Cache Block
22	10 110	Miss	110

Cache Example

Cache

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem(11010)
011	N		
100	N		
101	N		
110	Y	10	Mem(10110)
111	N		

Look-ups

Dec Word Addr	Binary Addr	Hit/Miss	Cache Block
22	10 110	Miss	110
26	11 010	Miss	010

Cache Example

Cache

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem(11010)
011	N		
100	N		
101	N		
110	Y	10	Mem(10110)
111	N		

Look-ups

Dec Word Addr	Binary Addr	Hit/Miss	Cache Block
22	10 110	Miss	110
26	11 010	Miss	010
22	10 110	Hit	110
26	11 010	Hit	010

Cache Example

Cache

Index	V	Tag	Data
000	Y	10	Mem(10000)
001	N		
010	Y	11	Mem(11010)
011	Y	00	Mem(00011)
100	N		
101	N		
110	Y	10	Mem(10110)
111	N		

Look-ups

Dec Word Addr	Binary Addr	Hit/Miss	Cache Block
22	10 110	Miss	110
26	11 010	Miss	010
22	10 110	Hit	110
26	11 010	Hit	010
16	10 000	Miss	000
3	00 011	Miss	011
16	10 000	Hit	000

Cache Example

Cache

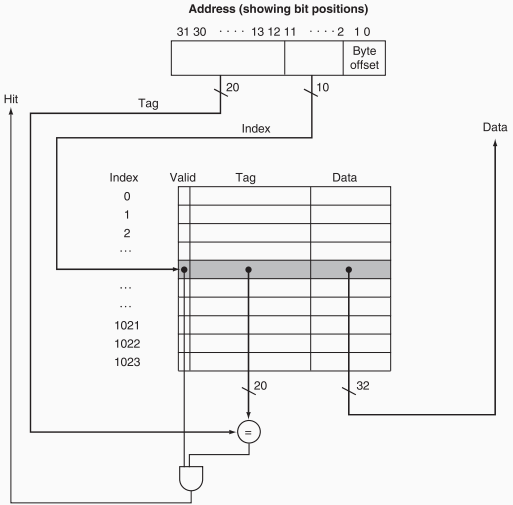
Index	V	Tag	Data
000	Y	10	Mem(10000)
001	N		
010	Y	10	Mem(10010)
011	Y	00	Mem(00011)
100	N		
101	N		
110	Y	10	Mem(10110)
111	N		

Look-ups

Dec Word Addr	Binary Addr	Hit/Miss	Cache Block
22	10 110	Miss	110
26	11 010	Miss	010
22	10 110	Hit	110
26	11 010	Hit	010
16	10 000	Miss	000
3	00 011	Miss	011
16	10 000	Hit	000
18	10 010	Miss	010

Address Subdivision

valid | tag | data



Total Bits in a Cache

- Total cache size depends on the size of the data and the address

Total Bits in a Cache

- Total cache size depends on the size of the data and the address
- Cache must store data, tags, and valid bits

Total Bits in a Cache

- Total cache size depends on the size of the data and the address
- Cache must store data, tags, and valid bits
- For a direct-mapped cache with 32-bit addresses:
  - Cache size is 2^n blocks (n bits for the index)
  - Block size is 2^m words (m bits for word offset, 2 for byte part)

we are accessing memory at byte level not word level so we also need 2 extra bits to specify which byte to select from the selected word in the selected cache line

Total Bits in a Cache

- Total cache size depends on the size of the data and the address
- Cache must store data, tags, and valid bits
- For a direct-mapped cache with 32-bit addresses:
  - Cache size is 2^n blocks (n bits for the index)
  - Block size is 2^m words (m bits for word offset, 2 for byte part)
- Tag field size is $32 - (n + m + 2)$ bits

Total Bits in a Cache

- Total cache size depends on the size of the data and the address
- Cache must store data, tags, and valid bits
- For a direct-mapped cache with 32-bit addresses:
  - Cache size is 2^n blocks (n bits for the index)
  - Block size is 2^m words (m bits for word offset, 2 for byte part)
- Tag field size is $32 - (n + m + 2)$ bits
- Total bits equal

$$2^n \times (2^m \times 32 + (32 - n - m - 2) + 1) = 2^n \times (2^m \times 32 + 31 - n - m)$$

Total Bits in a Cache

- Total cache size depends on the size of the data and the address
- Cache must store data, tags, and valid bits
- For a direct-mapped cache with 32-bit addresses:
  - Cache size is 2^n blocks (n bits for the index)
  - Block size is 2^m words (m bits for word offset, 2 for byte part)
- Tag field size is $32 - (n + m + 2)$ bits
- Total bits equal

  $$2^n \times (2^m \times 32 + (32 - n - m - 2) + 1) = 2^n \times (2^m \times 32 + 31 - n - m)$$
- Naming conventions specify cache size based only on the data portion

```
4 words per block
2^2 words ->
00 -> word1
01 -> word2
10 -> word3
11 -> word4

00 -> byte1
01 -> byte2
10 -> byte3
11 -> byte4
```

Example: Bits in a Cache

- Task: Find total bits for a 16 KiB direct-mapped cache with 64-bit addresses

Example: Bits in a Cache

- Task: Find total bits for a 16 KiB direct-mapped cache with 64-bit addresses
- 16 KiB of data is equivalent to 4096 (2^{12}) words

Example: Bits in a Cache

- Task: Find total bits for a 16 KiB direct-mapped cache with 64-bit addresses
- 16 KiB of data is equivalent to 4096 (2^{12}) words
- Four-word blocks (2^2) mean there are 1024 (2^{10}) blocks in the cache

Example: Bits in a Cache

- Task: Find total bits for a 16 KiB direct-mapped cache with 64-bit addresses
- 16 KiB of data is equivalent to 4096 (2^{12}) words
- Four-word blocks (2^2) mean there are 1024 (2^{10}) blocks in the cache
- Each block has $4 \times 32 = 128$ bits of data

Example: Bits in a Cache

- Task: Find total bits for a 16 KiB direct-mapped cache with 64-bit addresses
- 16 KiB of data is equivalent to 4096 (2^{12}) words
- Four-word blocks (2^2) mean there are 1024 (2^{10}) blocks in the cache
- Each block has $4 \times 32 = 128$ bits of data
- Tag size is $64 - 10$ (index) $- 2$ (word) $- 2$ (byte) $= 50$ bits

Example: Bits in a Cache

- Task: Find total bits for a 16 KiB direct-mapped cache with 64-bit addresses
- 16 KiB of data is equivalent to 4096 (2^{12}) words
- Four-word blocks (2^2) mean there are 1024 (2^{10}) blocks in the cache
- Each block has $4 \times 32 = 128$ bits of data
- Tag size is $64 - 10$ (index) $- 2$ (word) $- 2$ (byte) $= 50$ bits
- Complete cache size is $2^{10} \times (128 + 50 + 1) =$ 179 Kibibits

Example: Bits in a Cache

- Task: Find total bits for a 16 KiB direct-mapped cache with 64-bit addresses
- 16 KiB of data is equivalent to 4096 (2^{12}) words
- Four-word blocks (2^2) mean there are 1024 (2^{10}) blocks in the cache
- Each block has $4 \times 32 = 128$ bits of data
- Tag size is $64 - 10$ (index) $- 2$ (word) $- 2$ (byte) $= 50$ bits
- Complete cache size is $2^{10} \times (128 + 50 + 1) = 179$ Kibibits
- Total physical size is roughly 1.4 times the required data storage

```
1 kibibit = 1024
bit (2^10 bits)
```

```
cache line-
1 + 50 + 128
179 / 128 = 1.398
```

Example: Mapping an Address

- Task: Map byte address 1200 in a 64-block cache with 16-byte blocks

Example: Mapping an Address

- Task: Map byte address 1200 in a 64-block cache with 16-byte blocks
- Block address equals $\lfloor$Byte address/Bytes per block$\rfloor$

Example: Mapping an Address

- Task: Map byte address 1200 in a 64-block cache with 16-byte blocks
- Block address equals $\lfloor \text{Byte address}/\text{Bytes per block} \rfloor$
- Block address for 1200 is $\lfloor 1200/16 \rfloor = 75$

Example: Mapping an Address

- Task: Map byte address 1200 in a 64-block cache with 16-byte blocks
- Block address equals $\lfloor \text{Byte address}/\text{Bytes per block} \rfloor$
- Block address for 1200 is $\lfloor 1200/16 \rfloor = 75$
- Cache block number equals the block address modulo total cache blocks

Example: Mapping an Address

- Task: Map byte address 1200 in a 64-block cache with 16-byte blocks
- Block address equals $\lfloor \text{Byte address}/\text{Bytes per block} \rfloor$
- Block address for 1200 is $\lfloor 1200/16 \rfloor = 75$
- Cache block number equals the block address modulo total cache blocks
- Cache block maps to $75 \textbf{ mod } 64 = 11$

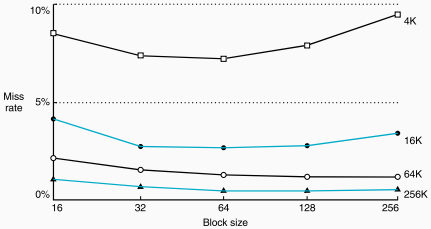
Example: Mapping an Address

- Task: Map byte address 1200 in a 64-block cache with 16-byte blocks
- Block address equals $\lfloor \text{Byte address}/\text{Bytes per block} \rfloor$
- Block address for 1200 is $\lfloor 1200/16 \rfloor = 75$
- Cache block number equals the block address modulo total cache blocks
- Cache block maps to 75 **mod** 64 = 11
- This specific block holds all addresses between 1200 and 1215

word1 1200 - 1203
word2 1204 - 1207
word3 1208 - 1211
word4 1212 - 1215

Block Size Trade-offs

- Larger blocks lower miss rates by exploiting spatial locality

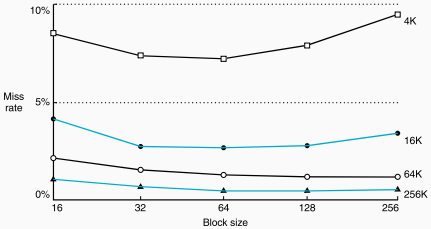


Miss rate versus block size

(each line represents a different sized cache)

Block Size Trade-offs

- Larger blocks lower miss rates by exploiting spatial locality
- Miss rates eventually increase if blocks become too large relative to the cache

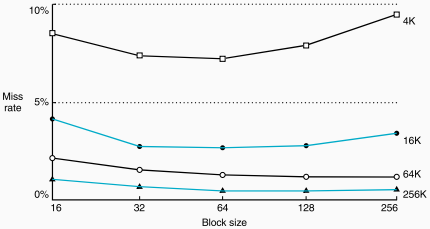


Miss rate versus block size

(each line represents a different sized cache)

Block Size Trade-offs

- Larger blocks lower miss rates by exploiting spatial locality
- Miss rates eventually increase if blocks become too large relative to the cache
  - Fewer total blocks cause heavy competition
  - Blocks get replaced before their contents are fully utilized

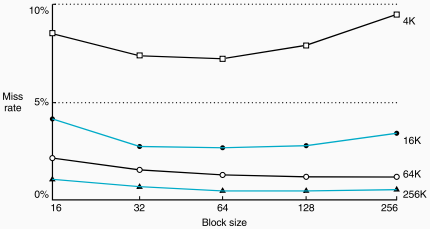


Miss rate versus block size

(each line represents a different sized cache)

Block Size Trade-offs

- Larger blocks lower miss rates by exploiting spatial locality
- Miss rates eventually increase if blocks become too large relative to the cache
  - Fewer total blocks cause heavy competition
  - Blocks get replaced before their contents are fully utilized
- Increasing block size also increases the miss penalty due to longer transfer times



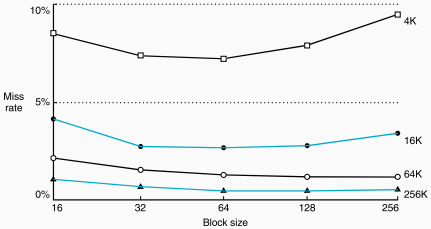
Miss rate versus block size

(each line represents a different sized cache)

Block Size Trade-offs

```
cache can hold N bits
if i increase block size i can hold less
blocks
```

- Larger blocks lower miss rates by exploiting spatial locality
- Miss rates eventually increase if blocks become too large relative to the cache
  - Fewer total blocks cause heavy competition
  - Blocks get replaced before their contents are fully utilized
- Increasing block size also increases the miss penalty due to longer transfer times
- Performance degrades when the miss penalty increase overwhelms the miss rate decrease



Miss rate versus block size

(each line represents a different sized cache)

Handling Cache Misses

- Control unit must detect miss and fetch data from memory

Handling Cache Misses

- Control unit must detect miss and fetch data from memory
- Hits are handled trivially, but misses require pipeline stall

Handling Cache Misses

- Control unit must detect miss and fetch data from memory
- Hits are handled trivially, but misses require pipeline stall
- Stall freezes temporary and programmer-visible registers

Handling Cache Misses

- Control unit must detect miss and fetch data from memory
- Hits are handled trivially, but misses require pipeline stall
- Stall freezes temporary and programmer-visible registers
- Process assumes in-order execution stalling on misses

Handling Cache Misses

- Control unit must detect miss and fetch data from memory
- Hits are handled trivially, but misses require pipeline stall
- Stall freezes temporary and programmer-visible registers
- Process assumes in-order execution stalling on misses
- Separate controller initiates memory access and refills cache

Hiding Transfer Time

- Transfer times can be partially hidden to reduce the effective miss penalty

Hiding Transfer Time

- Transfer times can be partially hidden to reduce the effective miss penalty
- Early restart resumes execution right after the requested word is returned
  - Works exceptionally well for sequential instruction accesses
  - Less effective for unpredictable data accesses

Hiding Transfer Time

```
cpu gives the required mem address and asks for
word2
early restart - word1 - word2 (cpu starts) - word3
- word4
CWF + Early Restart - word2 (cpu starts) - word3 -
word4 - word1
```

- Transfer times can be partially hidden to reduce the effective miss penalty
- Early restart resumes execution right after the requested word is returned
  - Works exceptionally well for sequential instruction accesses
  - Less effective for unpredictable data accesses
- Critical word first transfers the requested word before the rest of the block
  - Remaining words are transferred sequentially and wrap around to the beginning
  - Slightly faster than early restart but faces identical limitations

early restart- cpu starts working immediately after the requested word is retrieved, doest wait for whole block
critical word first- requested block is transferred first

Instruction Cache Miss Steps

- Instruction miss renders Instruction Register invalid

Instruction Cache Miss Steps

- Instruction miss renders Instruction Register invalid
- Target address is calculated as $PC - 4$

Instruction Cache Miss Steps

- Instruction miss renders Instruction Register invalid
- Target address is calculated as $PC - 4$
- Execution steps on miss:
  1. Send original PC value to memory
  2. Instruct memory to read and wait for access completion
  3. Write cache entry: fill data, update tag, set valid bit
  4. Restart instruction execution to refetch from cache

Instruction Cache Miss Steps

- Instruction miss renders Instruction Register invalid
- Target address is calculated as $PC - 4$
- Execution steps on miss:
  1. Send original PC value to memory
  2. Instruct memory to read and wait for access completion
  3. Write cache entry: fill data, update tag, set valid bit
  4. Restart instruction execution to refetch from cache
- Data miss handling is essentially identical

Handling Writes: Consistency

- Writing only to cache makes it inconsistent with memory

Handling Writes: Consistency

- Writing only to cache makes it inconsistent with memory
- Write-through: Update both cache and main memory simultaneously

Handling Writes: Consistency

```
cpu requests memA
memA gets written to the cache
cpu may change the value of memA
the value of new memA is written in cache
need to ensure to update the original value of
memA in the RAM
```

- Writing only to cache makes it inconsistent with memory
- Write-through: Update both cache and main memory simultaneously
- Write miss procedure in write-through:
  - Fetch block from memory
  - Overwrite specific word in cache
  - Write word to main memory

Write-Through Performance

- Direct writes to memory are slow (e.g., 100+ cycles)

Write-Through Performance

- Direct writes to memory are slow (e.g., 100+ cycles)
- High store frequency can drastically reduce performance

Write-Through Performance

- Direct writes to memory are slow (e.g., 100+ cycles)
- High store frequency can drastically reduce performance
- Solution: Use a write buffer

Write-Through Performance

- Direct writes to memory are slow (e.g., 100+ cycles)
- High store frequency can drastically reduce performance
- Solution: Use a write buffer
- Processor stores data in buffer and continues execution

Write-Through Performance

- Direct writes to memory are slow (e.g., 100+ cycles)
- High store frequency can drastically reduce performance
- Solution: Use a write buffer
- Processor stores data in buffer and continues execution
- Processor stalls only if write buffer is full

Write-Through Performance

- Direct writes to memory are slow (e.g., 100+ cycles)
- High store frequency can drastically reduce performance
- Solution: Use a write buffer
- Processor stores data in buffer and continues execution
- Processor stalls only if write buffer is full
- Multiple entries in buffer help handle write bursts

Write-Back Scheme

- Write-back: New value written only to cache block

Write-Back Scheme

- Write-back: New value written only to cache block
- Modified block written to memory only when replaced

Write-Back Scheme

- Write-back: New value written only to cache block
- Modified block written to memory only when replaced
- Improves performance if write generation exceeds memory speed

Write-Back Scheme

- Write-back: New value written only to cache block
- Modified block written to memory only when replaced
- Improves performance if write generation exceeds memory speed
- More complex to implement than write-through

Write Miss Policies

- Write allocate: fetch block from memory, then overwrite

Write Miss Policies

- Write allocate: fetch block from memory, then overwrite
- No write allocate: update memory directly without caching

Write Miss Policies

- Write allocate: fetch block from memory, then overwrite
- No write allocate: update memory directly without caching
- No write allocate useful when writing entire blocks (e.g., OS zeroing pages)

Write Miss Policies

- Write allocate: fetch block from memory, then overwrite
- No write allocate: update memory directly without caching
- No write allocate useful when writing entire blocks (e.g., OS zeroing pages)
- Policy can sometimes be changed on per-page basis

Implementing Write-Back

- Write-through can write and check tag in one cycle

Implementing Write-Back

- Write-through can write and check tag in one cycle
- Write-back requires confirming hit before writing to avoid data destruction

Implementing Write-Back

- Write-through can write and check tag in one cycle
- Write-back requires confirming hit before writing to avoid data destruction
- Implementation options:
  - Two-cycle process (check hit, then write)
  - Store buffer to pipeline access

Implementing Write-Back

```
cache line - line1 *line2* line3 line4
taking line2 in a store buffer, the line2 gets evicted by
the new memory. Then buffer updates the value for the old
line2 in the RAM.
```

- Write-through can write and check tag in one cycle
- Write-back requires confirming hit before writing to avoid data destruction
- Implementation options:
  - Two-cycle process (check hit, then write)
  - Store buffer to pipeline access
- Write-back buffer holds modified blocks during replacement to reduce penalty

Example: Intrinsity FastMATH

- Embedded MIPS processor
  - 12-stage pipeline
  - Instruction and data access on each cycle

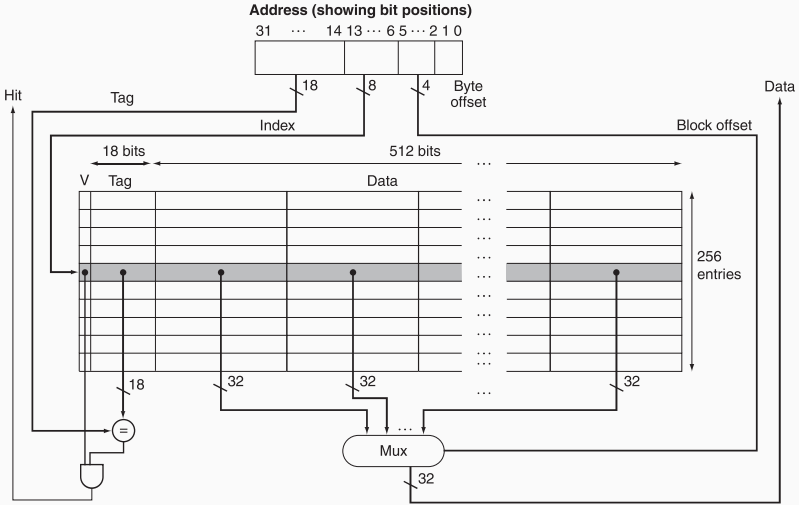
Example: Intrinsity FastMATH

- Embedded MIPS processor
  - 12-stage pipeline
  - Instruction and data access on each cycle
- Split cache: Separate I-cache and D-cache
  - Each 16KiB — 256 blocks $\times$ 16 words/block
  - D-cache offers both write-through and write-back, OS gets to decide the strategy

Example: Intrinsity FastMATH

- Embedded MIPS processor
  - 12-stage pipeline
  - Instruction and data access on each cycle
- Split cache: Separate I-cache and D-cache
  - Each 16KiB — 256 blocks $\times$ 16 words/block
  - D-cache offers both write-through and write-back, OS gets to decide the strategy
- SPEC2000 miss rate:
  - I-cache: 0.4%
  - D-cache: 11.4%
  - Effective combined (weighted average): 3.2%

Example: Intrinsity FastMATH Cache Organization



Cache Summary and Memory Performance

- Direct-mapped cache uses simple hit/miss logic

Cache Summary and Memory Performance

- Direct-mapped cache uses simple hit/miss logic
- Write policies ensure consistency:
  - Write-through: updates memory on every write
  - Write-back: updates memory only on replacement

Cache Summary and Memory Performance

- Direct-mapped cache uses simple hit/miss logic
- Write policies ensure consistency:
  - Write-through: updates memory on every write
  - Write-back: updates memory only on replacement
- Larger blocks exploit spatial locality to reduce miss rate

Cache Summary and Memory Performance

- Direct-mapped cache uses simple hit/miss logic
- Write policies ensure consistency:
  - Write-through: updates memory on every write
  - Write-back: updates memory only on replacement
- Larger blocks exploit spatial locality to reduce miss rate
- Increased block size risks higher miss penalty

Cache Summary and Memory Performance

- Direct-mapped cache uses simple hit/miss logic
- Write policies ensure consistency:
  - Write-through: updates memory on every write
  - Write-back: updates memory only on replacement
- Larger blocks exploit spatial locality to reduce miss rate
- Increased block size risks higher miss penalty
- Bandwidth improvements needed for efficiency —
  - Wider memory and interleaving
  - Optimized DRAM interfaces for burst mode

```
in ssd, data is stored in pages
each page has a data bank
data is first transferred to data bank,
then transferred to RAM
data is spread across multiple banks to
reduce the load for each bank. So each
bank acts in parallel, increasing
throughput.
```

Measuring and Improving Cache Performance

Measuring Cache Performance

- CPU time can be divided into two components —
  - CPU execution clock cycles: Normally includes cache hit time
  - Memory-stall clock cycles: Mainly from cache misses

$$\boxed{\textit{CPU time} = (\textit{CPU execution cycles} + \textit{Memory-stall cycles}) \times \textit{Clock cycle time}}$$

read miss - cache miss (data required not in cache)
write miss - data to be written is not in cache or write buffer full

Measuring Cache Performance

- CPU time can be divided into two components —
  - CPU execution clock cycles: Normally includes cache hit time
  - Memory-stall clock cycles: Mainly from cache misses

$$\boxed{\textit{CPU time} = (\textit{CPU execution cycles} + \textit{Memory-stall cycles}) \times \textit{Clock cycle time}}$$

- With simplified assumptions, we can define memory-stall clock cycles as —

$$\boxed{\textit{Memory-stall cycles} = (\textit{Read-stall cycles} + \textit{Write-stall cycles})}$$

Where,

- $\textit{Read-stall cycles} = \frac{\textit{Reads}}{\textit{Program}} \times \textit{Read miss rate} \times \textit{Read miss penalty}$
- $\textit{Write-stall cycles} = \left(\frac{\textit{Writes}}{\textit{Program}} \times \textit{Write miss rate} \times \textit{Write miss penalty}\right) + \textit{Write buffer stalls}$

Memory-stall Clock Cycles

- Considering write buffer stalls are negligible, in most write-through organizations, the read and write miss penalties are same.

$$\begin{aligned}\therefore \textit{Memory-stall cycles} &= \frac{\textit{Memory access}}{\textit{Program}} \times \textit{Miss rate} \times \textit{Miss penalty} \\ &= \frac{\textit{Instructions}}{\textit{Program}} \times \frac{\textit{Misses}}{\textit{Instruction}} \times \textit{Miss penalty}\end{aligned}$$

Cache Performance Example

- Given,
  - I-cache miss rate = 2%
  - D-cache miss rate = 4%
  - Miss penalty is = 100 cycles
  - Base CPI without stalls (ideal cache) = 2
  - Load and stores are 36% of instructions

Cache Performance Example

- Given,
  - I-cache miss rate = 2%
  - D-cache miss rate = 4%
  - Miss penalty is = 100 cycles
  - Base CPI without stalls (ideal cache) = 2
  - Load and stores are 36% of instructions
- Solution:

Cache Performance Example

- Given,
  - I-cache miss rate = 2%
  - D-cache miss rate = 4%
  - Miss penalty is = 100 cycles
  - Base CPI without stalls (ideal cache) = 2
  - Load and stores are 36% of instructions
- Solution:
  - Memory miss cycles count in terms of I instructions:
    - I-cache: $I \times 2\% \times 100 = 2.00I$
    - D-cache: $I \times 36\% \times 4\% \times 100 = 1.44I$

Cache Performance Example

- Given,
  - I-cache miss rate $= 2\%$
  - D-cache miss rate $= 4\%$
  - Miss penalty is $= 100$ cycles
  - Base CPI without stalls (ideal cache) $= 2$
  - Load and stores are 36% of instructions
- Solution:
  - Memory miss cycles count in terms of I instructions:
    - I-cache: $I \times 2\% \times 100 = 2.00I$
    - D-cache: $I \times 36\% \times 4\% \times 100 = 1.44I$
  - Total stalled cycle count $= 2.00I + 1.44I = 3.44I$

Cache Performance Example

- Given,
  - I-cache miss rate $= 2\%$
  - D-cache miss rate $= 4\%$
  - Miss penalty is $= 100$ cycles
  - Base CPI without stalls (ideal cache) $= 2$
  - Load and stores are 36% of instructions
- Solution:
  - Memory miss cycles count in terms of I instructions:
    - I-cache: $I \times 2\% \times 100 = 2.00I$
    - D-cache: $I \times 36\% \times 4\% \times 100 = 1.44I$
  - Total stalled cycle count $= 2.00I + 1.44I = 3.44I$
  - Total CPI including memory stalls $= 2 + 3.44 = 5.44$

Cache Performance Example

- Given,
  - I-cache miss rate = 2%
  - D-cache miss rate = 4%
  - Miss penalty is = 100 cycles
  - Base CPI without stalls (ideal cache) = 2
  - Load and stores are 36% of instructions
- Solution:
  - Memory miss cycles count in terms of I instructions:
    - I-cache: $I \times 2\% \times 100 = 2.00I$
    - D-cache: $I \times 36\% \times 4\% \times 100 = 1.44I$
  - Total stalled cycle count $= 2.00I + 1.44I = 3.44I$
  - Total CPI including memory stalls $= 2 + 3.44 = 5.44$
  - Perfect cache/Ideal CPU is $5.44/2 = 2.72$ times faster

Cache Performance Example

- Given,
  - I-cache miss rate = 2%
  - D-cache miss rate = 4%
  - Miss penalty is = 100 cycles
  - Base CPI without stalls (ideal cache) = 2
  - Load and stores are 36% of instructions
- Solution:
  - Memory miss cycles count in terms of I instructions:
    - I-cache: $I \times 2\% \times 100 = 2.00I$
    - D-cache: $I \times 36\% \times 4\% \times 100 = 1.44I$
  - Total stalled cycle count $= 2.00I + 1.44I = 3.44I$
  - Total CPI including memory stalls $= 2 + 3.44 = 5.44$
  - Perfect cache/Ideal CPU is $^{5.44}/_{2} = 2.72$ times faster
  - If base CPI was 1, a perfect system cache would be $^{1+3.44}/_{1} = 4.44$ times faster

```
CPI(including stalls) =  CPI(without stalls) +
(No.of Stalled cyles / Instructions)

=> CPI(including stalls) = 2 + (3.44I/I)
=>CPI(including stalls) = 5.44

As the CPI of the CPU increases, the miss penalty
remains constant. Since the clock speed of the CPU
is greater, the number of stalls the CPU has to
undergo is greater.
```

Average Access Time

- Hit time is also an important important factor in performance measurement

Average Access Time

- Hit time is also an important important factor in performance measurement
- Designers sometimes use Average Memory Access Time (AMAT)

$$AMAT = Hit\ time + Miss\ rate \times Miss\ penalty$$

Average Access Time

- Hit time is also an important important factor in performance measurement
- Designers sometimes use Average Memory Access Time (AMAT)

$$AMAT = Hit\ time + Miss\ rate \times Miss\ penalty$$

Example

Find the AMAT for a processor with a 1 ns clock cycle time, a miss penalty of 20 clock cycles, a miss rate of 5%, and a cache access time (including hit detection) of 1 clock cycle. Assume that the read and write miss penalties are the same and ignore other write stalls.

Average Access Time

- Hit time is also an important important factor in performance measurement
- Designers sometimes use Average Memory Access Time (AMAT)

$$AMAT = Hit\ time + Miss\ rate \times Miss\ penalty$$

Example

Find the AMAT for a processor with a 1 ns clock cycle time, a miss penalty of 20 clock cycles, a miss rate of 5%, and a cache access time (including hit detection) of 1 clock cycle. Assume that the read and write miss penalties are the same and ignore other write stalls.

Solution:

$$AMAT = 1 + 0.05 \times 20 = 2ns\ (2 \text{ clock cycles})$$

Performance Summary

- Miss penalty becomes more significant with CPU performance increase

Performance Summary

- Miss penalty becomes more significant with CPU performance increase
- With decreasing base CPI, greater proportion of time is spent of memory stalls

Performance Summary

- Miss penalty becomes more significant with CPU performance increase
- With decreasing base CPI, greater proportion of time is spent of memory stalls
- Increasing clock rate (smaller clock period) makes memory stalls account for more CPU cycles

Performance Summary

- Miss penalty becomes more significant with CPU performance increase
- With decreasing base CPI, greater proportion of time is spent of memory stalls
- Increasing clock rate (smaller clock period) makes memory stalls account for more CPU cycles
- Cache behavior cannot be neglected when evaluating system performance

Associative Caches

Schemes other than direct mapping exist for block placement in a cache

Associative Caches

Schemes other than direct mapping exist for block placement in a cache

- Fully associative
  - Allows a given block to go in any cache entry
  - Requires all entries to be searched at once
  - Comparator associated with each cache entry for parallel search (expensive)

Associative Caches

Schemes other than direct mapping exist for block placement in a cache

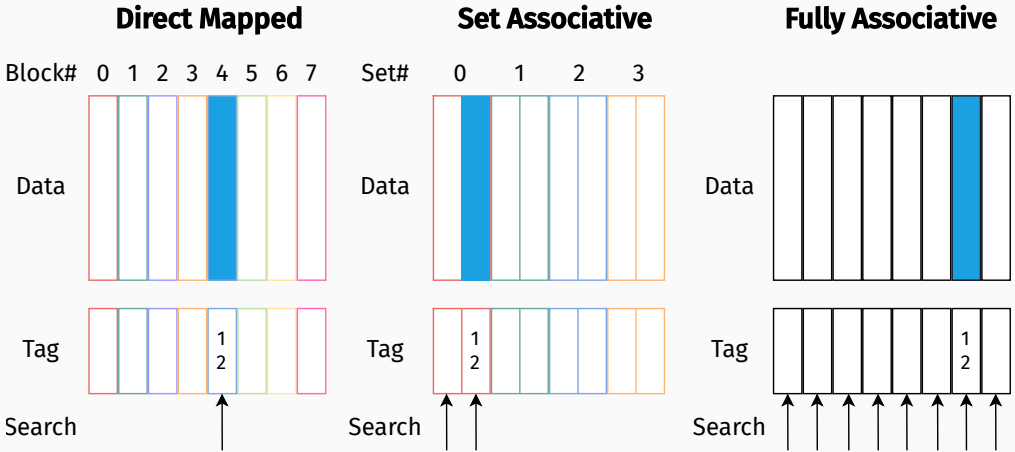
```
n-way where n=3
memA can go to LocationA(blockA1-blockA3)
```

- Fully associative
  - Allows a given block to go in any cache entry
  - Requires all entries to be searched at once
  - Comparator associated with each cache entry for parallel search (expensive)
- *n*-way set associative
  - Each block can be placed within a fixed number of locations (in a set)
  - Each set contains *n* entries
  - Block number and set count determines which set to put in

$$\textit{Block number} \mod \textit{Number of sets in the cache}$$

  - Search all the entries in a given set at once
  - *n* comparators (comparatively less expensive)

Associate Cache Example



Spectrum of Associativity

One-way set associative (direct mapped)

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Block	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Block	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

Associativity Example I

- Compare three small caches consisting of four one-word blocks —
  - Direct mapped, 2-way set associative, and fully associative
  - Block address access sequence: 0, 8, 0, 6, 8

Associativity Example I

- Compare three small caches consisting of four one-word blocks —
  - Direct mapped, 2-way set associative, and fully associative
  - Block address access sequence: 0, 8, 0, 6, 8
- Direct Mapped ($0 \mod 4 = 0$; $6 \mod 4 = 2$; $8 \mod 4 = 0$)

Address of memory block addressed	Hit or miss	Content of the cache blocks after reference			
		0	1	2	3
0	miss	Memory[0]			
8	miss	Memory[8]			
0	miss	Memory[0]			
6	miss	Memory[0]		Memory[6]	
8	miss	Memory[8]		Memory[6]	

Associativity Example II

- 2-way Set Associative ($0 \mod 2 = 0$; $6 \mod 2 = 0$; $8 \mod 2 = 0$)

Address of memory block addressed	Hit or miss	Content of the cache blocks after reference			
		Set 0	Set 0	Set 1	Set 1
0	miss	Memory[0]			
8	miss	Memory[0]	Memory[8]		
0	hit	Memory[0]	Memory[8]		
6	miss	Memory[0]	Memory[6]		
8	miss	Memory[8]	Memory[6]		

Associativity Example III

- Fully Associative

Address of memory block addressed	Hit or miss	Content of the cache blocks after reference			
		Block 0	Block 1	Block 2	Block 3
0	miss	Memory[0]			
8	miss	Memory[0]	Memory[8]		
0	hit	Memory[0]	Memory[8]		
6	miss	Memory[0]	Memory[8]	Memory[6]	
8	hit	Memory[0]	Memory[8]	Memory[6]	

How Much Associativity?

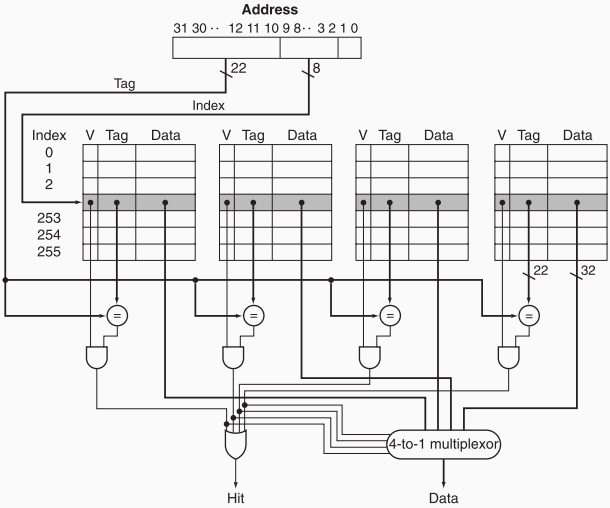
- Increased associativity decreases miss rate, but with diminishing returns

How Much Associativity?

- Increased associativity decreases miss rate, but with diminishing returns
- Simulation of a system with 64KiB D-cache, 16-word blocks (Intrinsity FastMATH) in SPEC2000 —

Associativity	Data Miss Rate
1	10.3%
2	8.6%
4	8.3%
8	8.1%

Set Associative Cache Organization



Replacement Policy

- If using direct mapped cache, there is no choice

Replacement Policy

- If using direct mapped cache, there is no choice
- For set associative,
  - Prefer non-valid entries if there is one (unused block)
  - Otherwise, choose among the entries in the set to replace

Replacement Policy

- If using direct mapped cache, there is no choice
- For set associative,
  - Prefer non-valid entries if there is one (unused block)
  - Otherwise, choose among the entries in the set to replace
- Least Recently Used (LRU) (most commonly used scheme)
  - Choose the one unused for the longest time
  - Simple for 2-way, manageable for 4-way, too complex/costly beyond that

Replacement Policy

- If using direct mapped cache, there is no choice
- For set associative,
  - Prefer non-valid entries if there is one (unused block)
  - Otherwise, choose among the entries in the set to replace
- Least Recently Used (LRU) (most commonly used scheme)
  - Choose the one unused for the longest time
  - Simple for 2-way, manageable for 4-way, too complex/costly beyond that
- Random
  - Gives approximately the same performance as LRU for high level of associativity

Multilevel Caches

- To close the gap between the fast processor clock rate and slow DRAM access, modern processors use at least one extra level of cache

Multilevel Caches

- To close the gap between the fast processor clock rate and slow DRAM access, modern processors use at least one extra level of cache
- Primary cache is attached to the CPU — small, but fast

Multilevel Caches

- To close the gap between the fast processor clock rate and slow DRAM access, modern processors use at least one extra level of cache
- Primary cache is attached to the CPU — small, but fast
- L2 cache services misses from the primary cache — larger, slower, but still faster than the main memory

Multilevel Caches

- To close the gap between the fast processor clock rate and slow DRAM access, modern processors use at least one extra level of cache
- Primary cache is attached to the CPU — small, but fast
- L2 cache services misses from the primary cache — larger, slower, but still faster than the main memory
- Main memory services L2 cache misses

Multilevel Caches

- To close the gap between the fast processor clock rate and slow DRAM access, modern processors use at least one extra level of cache
- Primary cache is attached to the CPU — small, but fast
- L2 cache services misses from the primary cache — larger, slower, but still faster than the main memory
- Main memory services L2 cache misses
- Most modern high-end systems (generally multi-core) include an L3 cache as well
  - L1 is private to a processor core, separate I- and D-caches
  - L2 also usually private to a processor core, generally stores both instruction and data
  - L3 is shared across the different cores

Multilevel Cache Example I

- Given,
  - CPU base CPI $= 1$, clock-rate $= 4GHz$
  - Miss rate/instruction $= 2\%$
  - Main memory access time $= 100ns$

Multilevel Cache Example I

- Given,
  - CPU base CPI $= 1$, clock-rate $= 4GHz$
  - Miss rate/instruction $= 2\%$
  - Main memory access time $= 100ns$
- Considering just the primary cache,
  - Miss penalty $= {}^{100ns}/_{0.25ns} = 400$ cycles (access time converted to cycles using clock-rate)
  - Effective CPI $= 1 + 2\% \times 400 = 9$

Multilevel Cache Example II

- Now consider an added L2 cache with,
  - Access time $= 5ns$
  - Global miss rate to main memory $= 0.5\%$
  - Main memory access time $= 100ns$

Multilevel Cache Example II

- Now consider an added L2 cache with,
  - Access time $= 5ns$
  - Global miss rate to main memory $= 0.5\%$
  - Main memory access time $= 100ns$
- Now, primary (L1) miss with L2 hit,
  - $\text{Penalty}_{L1 \rightarrow L2} = {}^{5ns}/_{0.25ns} = 20$ cycles

Multilevel Cache Example II

- Now consider an added L2 cache with,
  - Access time $= 5ns$
  - Global miss rate to main memory $= 0.5\%$
  - Main memory access time $= 100ns$
- Now, primary (L1) miss with L2 hit,
  - $\text{Penalty}_{L1 \rightarrow L2} = {}^{5ns}/_{0.25ns} = 20$ cycles
- And, primary (L1) miss with L2 miss,
  - $\text{Penalty}_{L2 \rightarrow Mem} = 400$ cycles (previously calculated)

Multilevel Cache Example II

- Now consider an added L2 cache with,
  - Access time $= 5ns$
  - Global miss rate to main memory $= 0.5\%$
  - Main memory access time $= 100ns$
- Now, primary (L1) miss with L2 hit,
  - $\text{Penalty}_{\text{L1}\rightarrow\text{L2}} = {}^{5ns}/_{0.25ns} = 20$ cycles
- And, primary (L1) miss with L2 miss,
  - $\text{Penalty}_{\text{L2}\rightarrow\text{Mem}} = 400$ cycles (previously calculated)
- Hence, effective CPI with L2 cache

$$= 1 + \text{Primary stalls per instruction} + \text{Secondary stalls per instruction}$$

$$= 1 + 2\% \times 20 + 0.5\% \times 400 = 3.4$$

Multilevel Cache Example II

- Now consider an added L2 cache with,
  - Access time $= 5ns$
  - Global miss rate to main memory $= 0.5\%$
  - Main memory access time $= 100ns$
- Now, primary (L1) miss with L2 hit,
  - $\text{Penalty}_{\text{L1}\rightarrow\text{L2}} = {}^{5ns}/_{0.25ns} = 20$ cycles
- And, primary (L1) miss with L2 miss,
  - $\text{Penalty}_{\text{L2}\rightarrow\text{Mem}} = 400$ cycles (previously calculated)
- Hence, effective CPI with L2 cache

  $= 1 + \text{Primary stalls per instruction} + \text{Secondary stalls per instruction}$

  $= 1 + 2\% \times 20 + 0.5\% \times 400 = 3.4$
- Performance ratio between without/with secondary cache $= {}^{9}/_{3.4} = 2.6$

Multilevel Cache Design Considerations

- Primary cache
  - Focus on minimal hit time

Multilevel Cache Design Considerations

- Primary cache
  - Focus on minimal hit time
- L2 cache
  - Focus on lowering miss rate to avoid main memory access
  - Hit time has comparatively lesser overall impact

Multilevel Cache Design Considerations

- Primary cache
  - Focus on minimal hit time
- L2 cache
  - Focus on lowering miss rate to avoid main memory access
  - Hit time has comparatively lesser overall impact
- Effect of the design considerations
  - L1 cache in a multilevel cache design is usually smaller than a single-cache system
  - L1 block size becomes smaller than L2 block size

Multilevel Cache Design Considerations

- Primary cache
  - Focus on minimal hit time
- L2 cache
  - Focus on lowering miss rate to avoid main memory access
  - Hit time has comparatively lesser overall impact
- Effect of the design considerations
  - L1 cache in a multilevel cache design is usually smaller than a single-cache system
  - L1 block size becomes smaller than L2 block size
- Considering L1 cache filters access, the local miss rate (miss in a specific cache level) of L2 cache can be quite high compared to the global miss rate (not found in any cache)

Interactions with Advanced CPUs

- Out-of-order CPUs can execute instructions during cache miss
  - Pending store stays in the load/store unit
  - Dependent instructions wait in reservation stations, while independent instructions continue

Interactions with Advanced CPUs

- Out-of-order CPUs can execute instructions during cache miss
  - Pending store stays in the load/store unit
  - Dependent instructions wait in reservation stations, while independent instructions continue
- Instead of instruction and data miss rates, misses per instruction is used

Interactions with Advanced CPUs

- Out-of-order CPUs can execute instructions during cache miss
  - Pending store stays in the load/store unit
  - Dependent instructions wait in reservation stations, while independent instructions continue
- Instead of instruction and data miss rates, misses per instruction is used
- Effect of miss depends on the program data flow — much harder to analyze and requires system simulations

Impact of Memory Hierarchy on Algorithmic Analysis

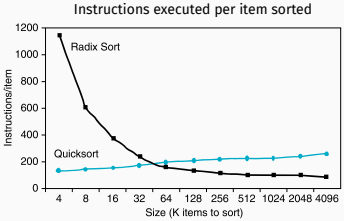
- Standard algorithmic analysis often ignores the impact of memory hierarchy

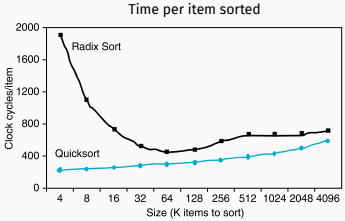
Impact of Memory Hierarchy on Algorithmic Analysis

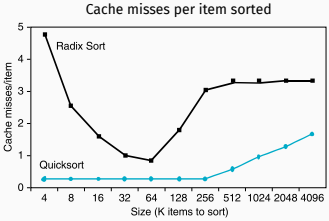
- Standard algorithmic analysis often ignores the impact of memory hierarchy
- While Radix Sort has advantage over Quicksort based on the number of instructions executed, Quicksort may still outperform Radix Sort if memory hierarchy is not taken into account

Impact of Memory Hierarchy on Algorithmic Analysis

- Standard algorithmic analysis often ignores the impact of memory hierarchy
- While Radix Sort has advantage over Quicksort based on the number of instructions executed, Quicksort may still outperform Radix Sort if memory hierarchy is not taken into account







Software Optimization via Blocking

- Memory hierarchy is crucial for program performance
- Software optimizations can drastically improve execution speed
- The main goal is to reuse data already in the cache
- This lowers miss rates by improving temporal locality

The Array Access Problem

```
a b c d e f
g h i j k l
m n o p q r
```

- Sequential memory access yields the best performance
- But some multi-dimensional arrays, like in the case of matrix multiplication, are accessed by rows and others by columns
- Storing arrays in row-major or column-major order does not fix this when both rows and columns are used in every loop iteration

What is Blocking?

```
A x B
a1  a2  a3  a4
a5  a6  a7  a8
a9  a10 a11 a12
a13 a14 a15 a16

A =
Ba1 Ba2
Ba3 Ba4

B =
Bb1 Bb2
Bb3 Bb4
```

- Instead of full rows or columns, we use submatrices or blocks
- Blocked algorithms operate purely on these smaller sections
- Maximizes accesses to data while it is loaded in the cache
- Keeps data active before it gets replaced by new memory loads

Standard Matrix Multiplication

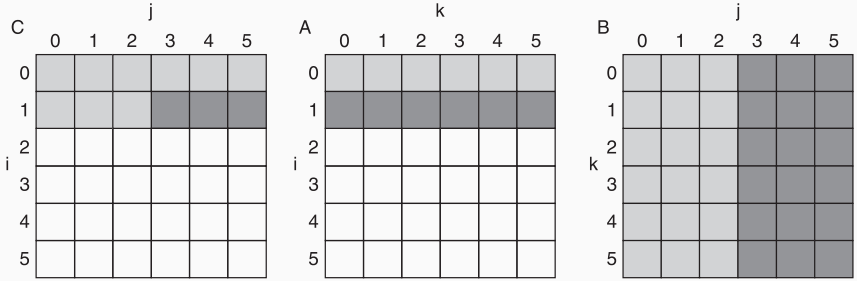
- Consider the multiplication of three $N \times N$ matrices ($C = A \times B$)
- The algorithm reads all elements of matrix B
- It reads the same elements in one row of matrix A repeatedly
- It writes to one row of matrix C

Standard Matrix Multiplication Code (Unblocked)

Inner loops for standard matrix multiplication —

```
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) { // Inner loop from here
        double cij = C[i+j*n]; /* cij = C[i][j] */
        for (int k = 0; k < n; k++ )
            cij += A[i+k*n] * B[k+j*n];
        C[i+j*n] = cij; /* C[i][j] = cij */
    }
}
```

Access Pattern in the Unblocked Code



A snapshot of the three arrays (matrices) C, A, and B when $N = 6$ and $i = 1$; the age of accesses to the array elements is indicated by shade: white means not yet touched, light means older accesses, and dark means newer accesses.

Cache Misses in Unblocked Code

- Number of capacity misses depends on N and cache size
- If the cache holds all three matrices, performance is optimal
- If the cache is too small, misses occur heavily for B and C
- Worst case requires $2N^3 + N^2$ memory words accessed

The Blocking Solution

- We change the code to compute on a submatrix
- The submatrix size is defined by a blocking factor
- The blocking factor is called BLOCKSIZE
- This ensures the accessed elements easily fit inside the cache

Blocked Matrix Multiplication Code

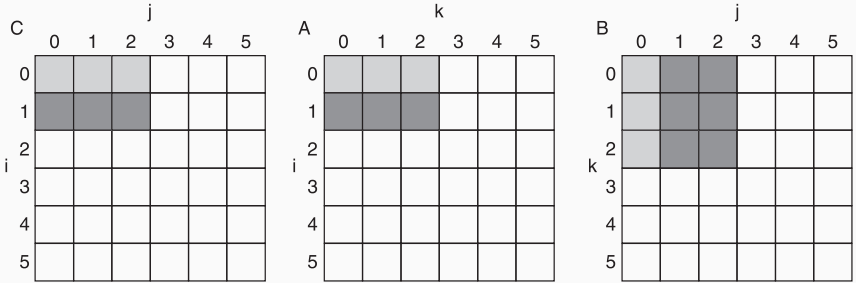
```
#define BLOCKSIZE 32

void do_block (int n, int si, int sj, int sk,  const double
    *A, const double *B, double *C) {
     for (int i = si; i < si+BLOCKSIZE; ++i)
         for (int j = sj; j < sj+BLOCKSIZE; ++j) {
             double cij = C[i+j*n];
             for(int k = sk; k < sk+BLOCKSIZE; k++)
                 cij += A[i+k*n] * B[k+j*n];
             C[i+j*n] = cij;
         }
}
```

Invoking the Blocked Code

The main function calls the block function repeatedly —

```
void dgemm (int n, const double* A, const double* B, double*
    C) {
    for (int sj = 0; sj < n; sj += BLOCKSIZE)
        for (int si = 0; si < n; si += BLOCKSIZE)
            for (int sk = 0; sk < n; sk += BLOCKSIZE)
                do_block(n, si, sj, sk, A, B, C);
}
```

Access Pattern in the Blocked Code



The age of accesses to the arrays C, A, and B when BLOCKSIZE $= 3$; in contrast to the unblocked version, fewer elements are accessed

Performance Improvements

- Total memory accessed becomes $2N^3/\text{BLOCKSIZE} + N^2$
- Performance improves roughly by a factor of BLOCKSIZE
- Matrix A benefits heavily from spatial locality
- Matrix B benefits heavily from temporal locality
- Small block sizes can also improve register allocation

Summary of Memory Hierarchy and Cache Optimization

- Memory Impact: Program execution time is heavily dictated by memory-stall cycles, which depend on miss rate and miss penalty

Summary of Memory Hierarchy and Cache Optimization

- Memory Impact: Program execution time is heavily dictated by memory-stall cycles, which depend on miss rate and miss penalty
- Reducing Miss Rates: Associative placement schemes provide flexibility in block location
  - Fully Associative: Highest flexibility but impractical due to high search costs
  - Set-Associative: A practical compromise using indexing to limit the search area

Summary of Memory Hierarchy and Cache Optimization

- Memory Impact: Program execution time is heavily dictated by memory-stall cycles, which depend on miss rate and miss penalty
- Reducing Miss Rates: Associative placement schemes provide flexibility in block location
  - Fully Associative: Highest flexibility but impractical due to high search costs
  - Set-Associative: A practical compromise using indexing to limit the search area
- Reducing Miss Penalties: Multilevel cache hierarchies ($L1, L2, \ldots$) bridge the gap between fast CPUs and slow main memory
  - Secondary caches are often $10\times$ larger than primary caches
  - Reduces penalty from > 100 cycles (main memory) to < 10 cycles (L2 cache)

Summary of Memory Hierarchy and Cache Optimization

- Memory Impact: Program execution time is heavily dictated by memory-stall cycles, which depend on miss rate and miss penalty
- Reducing Miss Rates: Associative placement schemes provide flexibility in block location
  - Fully Associative: Highest flexibility but impractical due to high search costs
  - Set-Associative: A practical compromise using indexing to limit the search area
- Reducing Miss Penalties: Multilevel cache hierarchies ($L1, L2, \ldots$) bridge the gap between fast CPUs and slow main memory
  - Secondary caches are often $10\times$ larger than primary caches
  - Reduces penalty from > 100 cycles (main memory) to < 10 cycles (L2 cache)
- Software Optimization: Algorithms can be restructured via blocking to improve temporal locality and cache effectiveness

Dependable Memory Hierarchy

Dependability and Failure

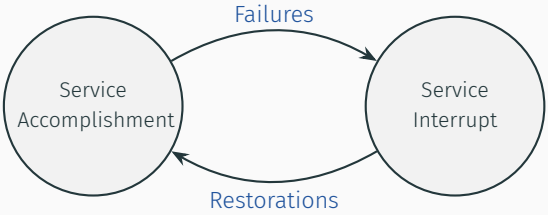
- Core idea for dependability is redundancy

Dependability and Failure

- Core idea for dependability is redundancy
- Systems alternate between two states —
  - Service accomplishment: service delivered as specified
  - Service interruption: delivered service differs from specification

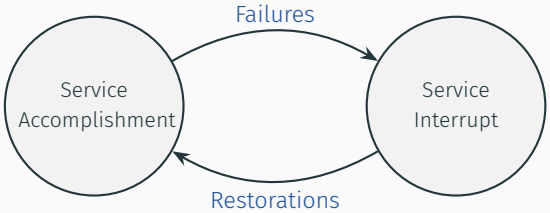
Dependability and Failure

- Core idea for dependability is redundancy
- Systems alternate between two states —
  - Service accomplishment: service delivered as specified
  - Service interruption: delivered service differs from specification
- Transitions between states —



Dependability and Failure

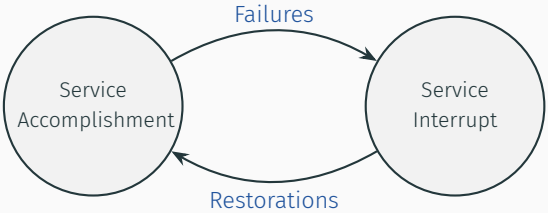
- Core idea for dependability is redundancy
- Systems alternate between two states —
  - Service accomplishment: service delivered as specified
  - Service interruption: delivered service differs from specification
- Transitions between states —



- Failures can be permanent or intermittent (harder to deal with)

Dependability and Failure

- Core idea for dependability is redundancy
- Systems alternate between two states —
  - Service accomplishment: service delivered as specified
  - Service interruption: delivered service differs from specification
- Transitions between states —



- Failures can be permanent or intermittent (harder to deal with)
- Two associated terms, reliability and availability

Reliability: MTTF and AFR

- Reliability measures continuous service accomplishment from a reference point

Reliability: MTTF and AFR

- Reliability measures continuous service accomplishment from a reference point
- MTTF (Mean Time To Failure) is the standard reliability measure

Reliability: MTTF and AFR

- Reliability measures continuous service accomplishment from a reference point
- MTTF (Mean Time To Failure) is the standard reliability measure
- AFR (Annual Failure Rate) is the percentage of devices expected to fail in a year

Reliability: MTTF and AFR

- Reliability measures continuous service accomplishment from a reference point
- MTTF (Mean Time To Failure) is the standard reliability measure
- AFR (Annual Failure Rate) is the percentage of devices expected to fail in a year
- Large MTTF values can be misleading

Reliability: MTTF and AFR

- Reliability measures continuous service accomplishment from a reference point
- MTTF (Mean Time To Failure) is the standard reliability measure
- AFR (Annual Failure Rate) is the percentage of devices expected to fail in a year
- Large MTTF values can be misleading
- AFR provides better intuition for large-scale systems

Example: MTTF vs AFR

- Assume disk MTTF is 1,000,000 hours (114 years)
- Assume warehouse-scale computer with 50,000 servers (100,000 disks)
- 1 year equals 8760 hours
- AFR is $^{8760}/_{1000000} = 0.876\%$
- Expected failures equal $100,000 \times 0.00876 = 876$ disks per year
- This averages to more than two disk failures per day

Availability and MTTR

- MTTR (Mean Time To Repair) measures service interruption

Availability and MTTR

- MTTR (Mean Time To Repair) measures service interruption
- MTBF (Mean Time Between Failures) is the sum of MTTF and MTTR

Availability and MTTR

- MTTR (Mean Time To Repair) measures service interruption
- MTBF (Mean Time Between Failures) is the sum of MTTF and MTTR
- Availability quantifies service accomplishment alternation

$$Availability = \frac{MTTF}{MTTF + MTTR}$$

Availability and MTTR

- MTTR (Mean Time To Repair) measures service interruption
- MTBF (Mean Time Between Failures) is the sum of MTTF and MTTR
- Availability quantifies service accomplishment alternation

$$Availability = \frac{MTTF}{MTTF + MTTR}$$

- Shrinking MTTR improves availability as much as increasing MTTF

Availability and MTTR

- MTTR (Mean Time To Repair) measures service interruption
- MTBF (Mean Time Between Failures) is the sum of MTTF and MTTR
- Availability quantifies service accomplishment alternation

$$Availability = \frac{MTTF}{MTTF + MTTR}$$

- Shrinking MTTR improves availability as much as increasing MTTF
- Availability is measured in nines (for example 5 nines is 99.999%, about 5 minutes of repair per year)

Faults and Improving MTTF

- Component failure is defined as a fault, which may not cause full system failure

Faults and Improving MTTF

- Component failure is defined as a fault, which may not cause full system failure
- Three ways to improve MTTF —
  1. Fault avoidance: prevent occurrence by construction
  2. Fault tolerance: use redundancy to allow service despite faults
  3. Fault forecasting: predict and replace before failure

Hamming Distance and Error Detection

- Invented by Richard Hamming (Turing Award, 1968)

Hamming Distance and Error Detection

- Invented by Richard Hamming (Turing Award, 1968)
- Hamming distance: minimum number of bits that differ between any two correct bit patterns

Hamming Distance and Error Detection

- Invented by Richard Hamming (Turing Award, 1968)
- Hamming distance: minimum number of bits that differ between any two correct bit patterns
- Example: distance between 0<u>1</u>1<u>0</u>11 and 0<u>0</u>1<u>1</u>11 is two

Hamming Distance and Error Detection

- Invented by Richard Hamming (Turing Award, 1968)
- Hamming distance: minimum number of bits that differ between any two correct bit patterns
- Example: distance between 0<u>1</u>1<u>0</u>11 and 0<u>0</u>1<u>1</u>11 is two
- A minimum distance of two allows detection of single-bit errors

Hamming Distance and Error Detection

- Invented by Richard Hamming (Turing Award, 1968)
- Hamming distance: minimum number of bits that differ between any two correct bit patterns
- Example: distance between 011011 and 001111 is two
- A minimum distance of two allows detection of single-bit errors
- Parity code counts the number of 1s to detect errors
  - Adds a parity bit to make the total number of 1s even (even parity)
  - Mismatch between stored parity and calculated parity signals an error

Example: Simple Parity

- Data: 31_{ten} or 00011111_{two} (five 1s)
- To achieve even parity, add a 1 as the parity bit: 000111111_{two}
- Single bit inversion (e.g., MSB flips)
  - Read as 100111111_{two} (seven 1s)
  - Odd parity calculated, but even parity expected: error detected
- Double bit inversion
  - Read as 110111111_{two} (eight 1s)
  - Even parity calculated: error not detected
- 1-bit parity cannot correct errors, only detect them

Hamming Error Correction Code (ECC)

- Minimum Hamming distance of 3 allows single bit error correction

Hamming Error Correction Code (ECC)

- Minimum Hamming distance of 3 allows single bit error correction
- Any single bit error is closer to the correct pattern than to any other valid pattern

Hamming Error Correction Code (ECC)

- Minimum Hamming distance of 3 allows single bit error correction
- Any single bit error is closer to the correct pattern than to any other valid pattern
- Bit numbering starts at 1 from the left

Hamming Error Correction Code (ECC)

- Minimum Hamming distance of 3 allows single bit error correction
- Any single bit error is closer to the correct pattern than to any other valid pattern
- Bit numbering starts at 1 from the left
- Parity bits are located at positions that are powers of 2 (1, 2, 4, 8, 16, ...)

Hamming Error Correction Code (ECC)

- Minimum Hamming distance of 3 allows single bit error correction
- Any single bit error is closer to the correct pattern than to any other valid pattern
- Bit numbering starts at 1 from the left
- Parity bits are located at positions that are powers of 2 (1, 2, 4, 8, 16, ...)
- Data bits occupy all other positions (3, 5, 6, 7, 9, 10, 11, 12...)

Hamming Error Correction Code (ECC)

- Minimum Hamming distance of 3 allows single bit error correction
- Any single bit error is closer to the correct pattern than to any other valid pattern
- Bit numbering starts at 1 from the left
- Parity bits are located at positions that are powers of 2 (1, 2, 4, 8, 16, ...)
- Data bits occupy all other positions (3, 5, 6, 7, 9, 10, 11, 12...)
- Parity bit positions determine the sequence of data bits they check —
  - Bit 1 checks positions where rightmost address bit is 1 (1, 3, 5, 7, 9, ...)
  - Bit 2 checks positions where second address bit is 1 (2, 3, 6, 7, 10, ...)
  - Bit 4 checks positions where third address bit is 1 (4–7, 12–15, ...)
  - Bit 8 checks positions where fourth address bit is 1 (8–15, 24–31, ...)

Hamming ECC Coverage Table

Bit Position		1	2	3	4	5	6	7	8	9	10	11	12
Encoded Data Bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity Bit Coverage	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

- Each data bit is covered by two or more parity bits

Hamming ECC Coverage Table

Bit Position		1	2	3	4	5	6	7	8	9	10	11	12
Encoded Data Bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity Bit Coverage	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

- Each data bit is covered by two or more parity bits
- Set parity bits to create even parity for each specific group

Hamming ECC Coverage Table

Bit Position		1	2	3	4	5	6	7	8	9	10	11	12
Encoded Data Bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity Bit Coverage	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

- Each data bit is covered by two or more parity bits
- Set parity bits to create even parity for each specific group
- To check for errors, evaluate parity for all groups

Hamming ECC Coverage Table

Bit Position		1	2	3	4	5	6	7	8	9	10	11	12
Encoded Data Bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity Bit Coverage	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

- Each data bit is covered by two or more parity bits
- Set parity bits to create even parity for each specific group
- To check for errors, evaluate parity for all groups
- If results are 0000, there is no error

Hamming ECC Coverage Table

Bit Position		1	2	3	4	5	6	7	8	9	10	11	12
Encoded Data Bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity Bit Coverage	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

- Each data bit is covered by two or more parity bits
- Set parity bits to create even parity for each specific group
- To check for errors, evaluate parity for all groups
- If results are 0000, there is no error
- If results form a non-zero binary number (e.g., $1010_{two} = 10_{ten}$), that number identifies the exact bit position of the error

Hamming ECC Coverage Table

Bit Position		1	2	3	4	5	6	7	8	9	10	11	12
Encoded Data Bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity Bit Coverage	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

- Each data bit is covered by two or more parity bits
- Set parity bits to create even parity for each specific group
- To check for errors, evaluate parity for all groups
- If results are 0000, there is no error
- If results form a non-zero binary number (e.g., $1010_{two} = 10_{ten}$), that number identifies the exact bit position of the error
- Correct the error by inverting that specific bit

Example: Hamming ECC Calculation

- Data byte: 10011010_{two}
- 12-bit pattern layout (p = parity, d = data): $p_1\ p_2\ d_1\ p_4\ d_2\ d_3\ d_4\ p_8\ d_5\ d_6\ d_7\ d_8$
- Initial state: $p_1\ p_2\ 1\ p_4\ 0\ 0\ 1\ p_8\ 1\ 0\ 1\ 0$
- Calculate parity bits for even parity
  - Position 1 (1,3,5,7,9,11): $\underline{p_1}\ p_2\ \underline{1}\ p_4\ \underline{0}\ 0\ \underline{1}\ p_8\ \underline{1}\ 0\ \underline{1}\ 0 \rightarrow$ four 1s $\rightarrow p_1 = 0$
  - Position 2 (2,3,6,7,10,11): $0\ \underline{p_2}\ \underline{1}\ p_4\ 0\ \underline{0}\ \underline{1}\ p_8\ 1\ \underline{0}\ \underline{1}\ 0 \rightarrow$ three 1s $\rightarrow p_2 = 1$
  - Position 4 (4,5,6,7,12): $0\ 1\ 1\ \underline{p_4}\ \underline{0}\ \underline{0}\ \underline{1}\ p_8\ 1\ 0\ 1\ \underline{0} \rightarrow$ one 1 $\rightarrow p_4 = 1$
  - Position 8 (8,9,10,11,12): $0\ 1\ 1\ 1\ 0\ 0\ 1\ \underline{p_8}\ \underline{1}\ \underline{0}\ \underline{1}\ \underline{0} \rightarrow$ two 1s $\rightarrow p_8 = 0$
- Final encoded word: 011100101010

Example: Correcting an Error

- Original word: 011100101010
- Assume bit 10 inverts during transmission: 011100101110
- Recalculate parities on received word
  - Parity 1 group: $\underline{0}$ 1 $\underline{1}$ 1 $\underline{0}$ 0 $\underline{1}$ 0 $\underline{1}$ $\underline{1}$ $\underline{1}$ 0 (four 1s) $\rightarrow$ even (OK)
  - Parity 2 group: 0 $\underline{1}$ $\underline{1}$ 1 0 $\underline{0}$ $\underline{1}$ 0 1 $\underline{1}$ $\underline{1}$ 0 (five 1s) $\rightarrow$ odd (error)
  - Parity 4 group: 0 1 1 $\underline{1}$ $\underline{0}$ $\underline{0}$ $\underline{1}$ 0 1 1 1 $\underline{0}$ (two 1s) $\rightarrow$ even (OK)
  - Parity 8 group: 0 1 1 1 0 0 1 $\underline{0}$ $\underline{1}$ $\underline{1}$ $\underline{1}$ $\underline{0}$ (three 1s) $\rightarrow$ odd (error)
- Parity bits 2 and 8 are incorrect
- $2 + 8 = 10$, so bit 10 is wrong
- Correct by inverting bit 10 back to 0

SEC/DED: Single Error Correcting / Double Error Detecting

- Increases minimum Hamming distance to 4 at the cost of one extra bit

SEC/DED: Single Error Correcting / Double Error Detecting

- Increases minimum Hamming distance to 4 at the cost of one extra bit
- Adds an overall parity bit (p_4 in a 4-bit data example) calculated over the entire word

SEC/DED: Single Error Correcting / Double Error Detecting

- Increases minimum Hamming distance to 4 at the cost of one extra bit
- Adds an overall parity bit (p_4 in a 4-bit data example) calculated over the entire word
- Decoding algorithm uses Hamming parity (H) and overall parity (P)
  - H even, P even: no error occurred
  - H odd, P odd: correctable single error
  - H even, P odd: single error occurred in the overall parity bit itself
  - H odd, P even: double error occurred (uncorrectable)

SEC/DED: Single Error Correcting / Double Error Detecting

- Increases minimum Hamming distance to 4 at the cost of one extra bit
- Adds an overall parity bit (p_4 in a 4-bit data example) calculated over the entire word
- Decoding algorithm uses Hamming parity (H) and overall parity (P)
  - H even, P even: no error occurred
  - H odd, P odd: correctable single error
  - H even, P odd: single error occurred in the overall parity bit itself
  - H odd, P even: double error occurred (uncorrectable)
- Standard in modern server memory (e.g., 72-bit DIMMs for 64-bit data)

Elaborations on Error Handling

- Parity Bit Calculation: For d data bits and p parity bits, $2^p \geq p + d + 1$

Elaborations on Error Handling

- Parity Bit Calculation: For d data bits and p parity bits, $2^p \geq p + d + 1$
- Chipkill (IBM) or SDDC (Intel)
  - Handles complete failure of a single memory chip
  - Distributes data and ECC like a RAID system
  - Reduces unrecoverable failures drastically compared to standard SEC/DED
  - Essential for warehouse-scale computers

Elaborations on Error Handling

- Parity Bit Calculation: For d data bits and p parity bits, $2^p \geq p + d + 1$
- Chipkill (IBM) or SDDC (Intel)
  - Handles complete failure of a single memory chip
  - Distributes data and ECC like a RAID system
  - Reduces unrecoverable failures drastically compared to standard SEC/DED
  - Essential for warehouse-scale computers
- Network Error Handling
  - Cyclic Redundancy Check (CRC) handles bursts of errors using binary division
  - Reed-Solomon codes use Galois fields and polynomials for complex multi-bit transmission errors

To be continued in *Large and Fast: Exploiting Memory Hierarchy: Part II*!